Developer 1 Guide
ENGINE

pack v2.3.5 (final) | TrueForge client, MCP tools, approval-gated proposals, cancel and failover

Forge-a-thon (TrueFoundry x Techfest IIT Bombay 2026-27). This guide replaces every earlier Developer 1 guide.

Every prompt in it is copied word for word from the pack's cards (`ai-guides/`). The pack is the source of truth.

Read this first (3 minutes)

You build everything that **talks to TrueForge**: the client, the local MCP tool server, the approval bridge, cancellation, retry and fallback. Developer 2 builds everything the user sees and decides what code the AI sees. You meet only at `src/types/contract.ts`. You do the thinking, steering and checking; your AI coding tool does the typing, **one small card at a time**.

Your scoring jobs

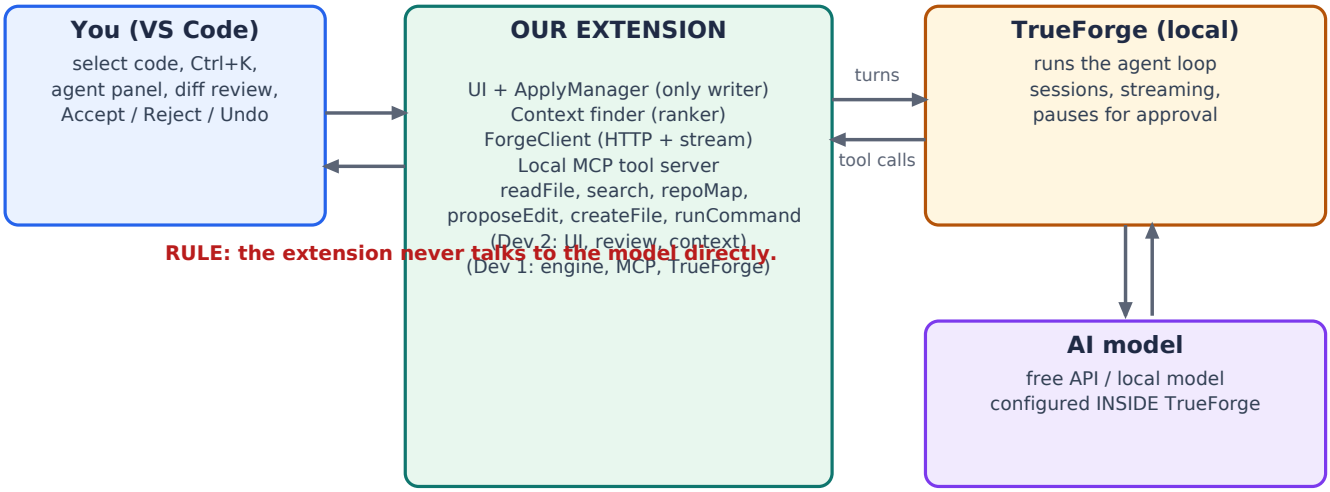
- **B4 TrueForge depth (10):** editor tools as MCP, TrueForge tool approval drives the review, one context-engineering feature visible in the session.
- **B3 Resilience (10):** automatic fallback to a second model, workspace unchanged (Task C).
- **A3 cancel (10) and A2 agent mode (10):** cancel leaves no half edits; the agent finds 3+ files through your capped tools.
- **B1 context (15):** you enforce the run budget and tool caps; the agent reports which files it chose and why.
- Also: README, scripts, write-up sections 1, 2, 3, 5, 7, 8.

> **Status on Wed 11 Nov 2026 = Day 8 of the 15-day plan**
> Nothing in the pack proves the TrueForge spike (S-02) has been run: there is no `docs/spike-report.md` and `fixtures/events/` is empty. **Gate G0 is open.** Today: finish setup (Part 2), run S-01, then S-02 together with Developer 2, and decide **ADR-2b** tonight if MCP registration, approval pause or session visibility fail. The re-baselined calendar is in Part 4. Submit by **Wed 18 Nov**; the hard deadline is Fri 20 Nov 23:59 IST.

Contents

Part	Title
1	Your half in plain words (concepts you must understand)
2	Setup today: tools, repo, worktree, AI tool
3	Free AI tools and models, and the rules for keys
4	Your re-baselined plan (Day 8 to Day 15)
5	Start here: S-01 bootstrap, S-02 spike, S-03 contract freeze
6	The build: cards E-01 to E-13
7	Handshake with Developer 2 and the integration moments
8	README and your write-up sections
9	Release, final checklists, live-call questions
10	When things go wrong

Part 1. Your half in plain words



The golden rule: the extension never calls a model provider. It talks only to TrueForge, and TrueForge talks to the model. Breaking this is an eligibility failure (the contest checks it first).

What you own, and what you never touch

You own	Never touch (stop and ask Developer 2)
`src/engine/`, `src/mcp/`, `forge/`, `scripts/`, `test/engine/`, `src/mocks/MockUiHost.ts`, `README.md`, root configs, `package.json` dependencies and scripts, docs 01-03, 06, 07 and write-up sections 1, 2, 3, 5, 7, 8	`src/extension.ts`, `src/ui/`, `src/review/`, `src/context/`, `media/`, `package.json` "contributes", `test/ui`, `test/context`, `test/e2e`
Shared (never alone): `src/types/contract.ts`, `fixtures/`. Change requests go in `docs/contract-change-requests.md`; joint PR.	**Docs owned by Dev 2:** 04, 05, write-up sections 4 and 6

Five ideas you must understand before writing code

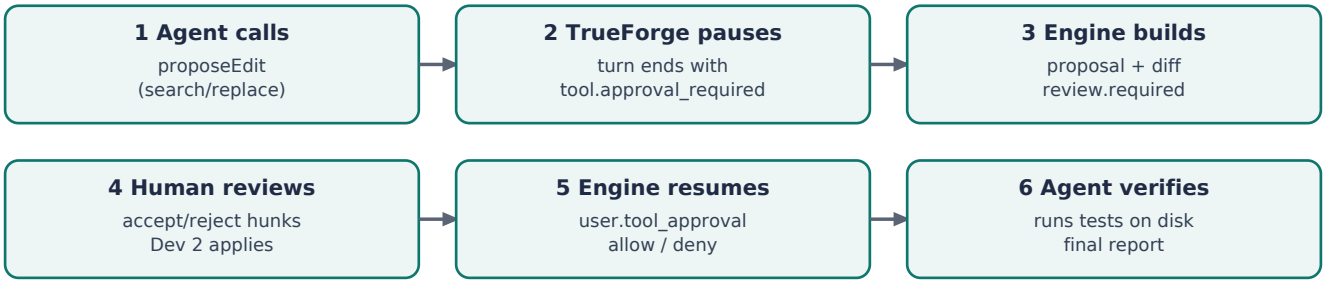
1. Agent, Session, Turn, Event

An **agent** is a saved definition (model + instructions + tools). A **session** is one conversation with it. A **turn** is one message you send and the agent's full reply. Everything the agent does streams back as **events** (tokens, tool requests, tool responses, "turn done"). Our contest "run" = one user request, which may span **several turns**.

2. MCP tools

MCP is a standard way to offer tools to an agent. Our extension runs a small **local MCP server** on `127.0.0.1` with tools: `repoMap`, `searchWorkspace`, `findSymbol`, `readFile`, `reportContext`, `proposeEdit`, `createFile`, `runCommand`. TrueForge's agent calls them; **the agent, not the extension, decides what to read and change.** Our tool server only protects itself with a token and size caps.

3. The approval pause (the heart of B4)



The agent **never writes**. `proposeEdit` only stages a proposal; TrueForge pauses; the human decides per hunk; Developer 2's ApplyManager is the only thing that writes; you then tell TrueForge **allow** (only if the change really landed) or **deny** with the true reason.

4. Truthful tool results

The agent plans from what you tell it. If the user rejected a hunk or the file was stale, the result must say so ("User rejected all changes to X", "File changed since proposal; re-read and re-propose"). Never answer "ok" because the user merely clicked Accept: `allow` is sent only after the apply was **verified**.

5. Correlation: matching the right human decision to the right tool call

- The engine knows each pending call by `sessionId:threadId:toolCallId` (the `tool.approval_required` event carries only ids, see F15 below).
- **Identical calls in one batch are grouped** into ONE proposal (`FileProposal.callIds` lists them all). One human decision applies to every id in the group, so order never matters.
- The MCP handler uses an exact call-id lookup if TrueForge passes the id (spike Q14); otherwise it looks up live records by `argsHash` = sha256(toolName, path, payload). Different decisions for the same hash = error + log `correlation_ambiguous`, **never a guess**. A denied call never reaches the handler, so you close its record when you send the deny.

Verified TrueForge facts you can rely on (from the SDK types; capture fixtures anyway)

Fact	What it means for you
F15 `tool.approval_required` = `{id, createdAt, threadId, toolCalls:[{id, sourceEventId}]}`: **no tool name, no arguments**	Read the tool name and arguments from the merged `model.message.toolCalls` via `sourceEventId`. This is the main path.
F16 approval is set **per MCP-server entry** in the agent: `requireApprovalForTools` takes `@all`, `@write`, `@destructive` or literal names; the default is `["@write","@destructive"]`	Set literal names `["proposeEdit","createFile"]` explicitly. Confirm the raw-HTTP field spelling in the spike.
F17 `preload` defaults to **false** (deferred tool discovery)	Set `preload: true` so weak or organiser-chosen models see the core tools without a discovery hop (Q17, ADR-13).
F18 `turn.done.state` is done / cancelled / error; a paused turn carries `requiredActions` and a paused status carries `actionRequiredOnEvents`	`mapTerminal` reads BOTH names and treats them the same.
F19 the input item `user.tool_approval_policy {type:"allow_session"}` would auto-allow a tool for the whole session	**Never send it** (it skips the human). CI greps for `tool_approval_policy` in `src`.
F20 `previous_turn_id` defaults to `auto`; `sessions.cancel` cancels the running last turn	Nothing special to do.
Local TrueForge default port is **8790** (8791 is the Docker Compose version)	Default `baseUrl` is `http://localhost:8790`; read the real URL from the startup output.
The SDK is **experimental** (breaking changes); types verified on 0.183.0 and re-checked on 0.184.1	**Pin exact versions** of `@truefoundry/trueforge` and `@truefoundry/trueforge-sdk`.

Failover and cancel outcomes (every row needs an automated test)

#	Situation	Outcome
1	Primary fails while streaming, nothing proposed	Retry by error class, then fail over and replay the original input on the fallback.
2	Primary fails after an edit was requested but before the pause is shown	Discard unreviewed calls (`proposal.discarded` "failover"), fail over, replay.
3	Failure while awaiting approval	A paused turn runs no model, so nothing fails during the wait; the resume turn can fail: treat as row 4 or 6.
4	Failure after at least one hunk was applied	**No replay** (it would apply twice): `run.failed` with a clear message; Undo stays available.
5	Fallback also fails	Exactly one `run.failed` with both causes; workspace unchanged if nothing was applied.
6	User cancels during back-off or failover	Timer cancelled, nothing replayed, `run.cancelled`, workspace unchanged.

Per-call states (`ToolCallState` in the contract): PROPOSED -> WAITING_FOR_REVIEW -> ACCEPTED or REJECTED -> APPLIED, APPLY_FAILED or STALE -> APPROVAL_SENT -> TOOL_ACKNOWLEDGED (or EXPIRED). `allow` is sent only from APPLIED; an illegal transition throws in tests.

Part 2. Setup today

Tools to install (about 45 minutes)

Tool	Check	Notes
Node.js 22.14 or newer	`node -v`	TrueForge needs >= 22.14. If you have 20.x (for example 20.19.0), **upgrade now** (nodejs.org LTS or nvm-windows) and reopen the terminal.
Git, **VS Code**, **Python 3.10+**	`git --version`, `code --version`, `python --version`	Set `git config --global user.name` and `user.email`.
GitHub account	log in	You create the repo; Developer 2 is added as collaborator.
Techfest registration + Team ID	saved	Registration closes **15 Nov**. Star github.com/truefoundry/trueforge and join the TrueForge Discord.
AI coding tool + a free model key	Part 3	Create keys yourself; paste them **only into TrueForge's UI**.

On native Windows, check early that `npx @truefoundry/trueforge@latest` starts (the repo documents WSL2 for source development). If it fails, use WSL2 and tell Developer 2.

Create the repo (once, about 15 minutes)

1. Unzip `forge-agent-pack-v2.3.5.zip`. Create a folder `forge-agent` and copy the **contents** of the `pack` folder into it (include hidden files: `.github`, `.gitignore`).
2. Edit `.github/CODEOWNERS`: replace `@DEV1_GITHUB` and `@DEV2_GITHUB` with your real GitHub usernames. (CI prints a warning until you do.)
3. On GitHub create a **Public** repo `forge-agent` with no README, license or .gitignore. Then run:

TERMINAL / CODE

```
git init && git add . && git commit -m "chore: import agent pack v2.3.5"
git branch -M main
git remote add origin https://github.com/YOUR_NAME/forge-agent.git
git push -u origin main
```

Add Developer 2 as collaborator; protect `main` (Settings -> Branches: require a pull request and passing CI). Then create your worktree: `git worktree add ../forge-dev1 -b dev1/s01-bootstrap main` and open it in its own VS Code window.

Run every card the same way

1. Open your own worktree in its own VS Code window; start your AI tool from that folder in a **fresh** session.
2. Paste the **Session start** prompt (above). Wait for its 6-line summary.
3. Paste the card's prompt (the boxed text in this guide; it is word for word the pack's card, followed by the standard footer). Add the excerpts the card asks for (spike report, fixtures).
4. The tool answers with a **plan**. Check: only your folders? tests included? any invented TrueForge fact? Reply `go` or correct it.
5. It implements and runs `npm run check`. **You run the thing yourself.** "It compiles" is not "it works".
6. `git diff --stat`, revert anything outside your folders, commit with the card's message, push, open a PR. Your teammate reviews against "Done when".

PASTE FIRST IN EVERY FRESH AI SESSION

```
Read AGENTS.md, docs/00-PROJECT-BIBLE.md, docs/01-ARCHITECTURE.md and src/types/contract.ts.
I am Developer 1. Summarise in 6 lines: my owned paths, the hard rules, and the card I am about to
  give you.
Then wait. Do not edit anything yet.
```

The routine in commands

TERMINAL / CODE

```
git worktree add ../forge-devN -b devN/<card> main   # once per card (N = 1 or 2)
cd ../forge-devN && npm ci            # node_modules are per worktree
# ... run the card in your AI tool ...
git status && git diff --stat         # anything outside YOUR folders? -> git restore <file>
git add <only the files you meant to change>
git commit -m "<message from the card>"
git push -u origin devN/<card>        # then open a PR; teammate reviews against "Done when"
git fetch origin && git rebase origin/main   # after each merge
```

Hard rules your AI tool must obey (AGENTS.md)

1) No direct model-provider calls. 2) Only `ApplyManager`/`UndoManager` write workspace files; `src/engine` and `src/mcp` never import fs write APIs or `vscode.workspace.applyEdit` (CI greps). 3) Tools never write. 4) No shell strings: `{file,args[]}` with `execFile`, `shell:false`. 5) **No task-specific words** (`get_debug_flag`, `cli.py`, `flask config`) in prompts, ranker, tools or production tests. 6) No secrets in logs or commits. 7) Never edit the contract alone. 8) Streams never throw; a run ends with exactly one terminal event. 9) Webview `textContent` only. 10) TypeScript strict, no `any` without a reason. 11) No new dependencies without the human's OK (you own `package.json` deps). 12) TrueForge facts only from `docs/02-TRUEFORGE-FACTS.md` and `docs/spike-report.md`; if a fact is UNVERIFIED and your task needs it, **stop and say so**.

Part 3. Free AI tools and models

There are **two different jobs** for AI. Do not mix them up. Facts below were read from public pages on 5-11 Nov 2026; free limits change often, so re-check before relying on one.

Job	What it is	Where you set it up
1. The tool that writes our code	A terminal or editor assistant that edits files for you. Any tool is allowed. Declare it in the write-up (it costs no points).	Your computer, your own worktree.
2. The model inside TrueForge	The "brain" the finished extension uses in the demo. Must be configured **inside TrueForge**. Two different providers: primary and fallback.	TrueForge UI -> model providers. Keys live only there.

Job 1: free tools that write the code

Tool	Free access (as found)	Catches
Freebuff (terminal agent). `npm install -g freebuff`, then run `freebuff` inside your worktree.	No subscription or key; ad-supported. Outside full-access regions about 5 h/day. Several free models.	May use prompts and code to personalise ads: keep its cloud/GitHub-repo connection OFF; some models say "may train on data"; availability depends on capacity.
Antigravity CLI (Google)	Replaced the free Gemini CLI on 18 Jun 2026; sign in with a Google account. No official free quota published; community reports a small starter quota.	The old Gemini CLI no longer serves free Google sign-ins. Test it on day one before relying on it.
GitHub Copilot (VS Code)	Free plan about 2,000 completions + 50 chat requests/month (third-party summary). Students can apply to GitHub Education for the Student plan.	Small monthly chat count; check GitHub's plans page.
Free web chat (Claude.ai, ChatGPT, Gemini)	Free accounts with message limits.	Copy-paste only, no file access. Great for explaining errors and reviewing a diff you paste.
Cline / Continue / Roo Code / opencode	Open source; bring your own key or a local model.	They have their own agent loops: fine as helpers, but never copy their loop into our extension (eligibility).
Claude Code	Needs a paid plan or API credit.	Not free. Good for E-06 and U-03 if one of you has it.

Job 2: free models to put inside TrueForge

Provider	Free tier (as found)	How	Catches
Google AI Studio (Gemini API key)	No card. Roughly 5-15 req/min and 20-1,500/day depending on model; Google cites 250/day for Flash on an unpaid key.	Catalog provider in TrueForge	HTTP 429 on per-minute caps; free-tier prompts may be used for training outside EU/UK/EEA: public Flask code only, never secrets.
Groq	No card. About 30 req/min, about 1,000/day; Llama-70B-class; 128K context.	OpenAI-compatible `https://api.groq.com/openai/v1`	Pick a model that supports tool calling.
OpenRouter	Free models about 20 req/min but only 50/day (about 1,000/day after a $10 top-up).	`https://openrouter.ai/api/v1`	Tiny daily cap: use as the fallback.
Cerebras	About 1M tokens/day, about 30 req/min.	OpenAI-compatible	Alternative.
Ollama (local)	Free; 7-8B models need about 16 GB RAM.	`http://localhost:11434/v1` (confirm in Ollama docs)	Weak at tool calls: use for the "weak model" drill only.

Rules for every AI tool and key

1) **Never paste a key, token or personal data into any AI tool**, a commit, a screenshot or the video. 2) Keys live only inside TrueForge (UI -> model providers). 3) If a key leaks: delete it at the provider first, then make a new one. 4) Primary and fallback must be **different providers**, otherwise Task C does not show a real switch. 5) Free tools have small windows and daily limits: one card per fresh session, commit after every card, keep a request tally in `docs/rehearsal-log.md`, and use `MockEngine`/fixtures instead of the live model whenever you can.

Handy prompts

When	Paste this
Plan first	Do not edit anything yet. Give me a plan: files to create or change, order, tests, and what could go wrong. Wait for my "go".
Tests first	Write the tests first and show me they fail for the right reason. Then implement until they pass. Do not change a test to make it pass.
Error	Here is the error: <paste exact text>. Explain the cause in 3 lines in simple words, propose the smallest fix, and do not refactor unrelated code.
Review before commit	Review my uncommitted diff. List: files outside my ownership, any hardcoded key/path/task word, any direct provider call, any workspace write outside ApplyManager/UndoManager, any shell string, any missing test. Be strict.
Learn	Explain <concept> to me like I am 12, then show a 10-line example from this project and name the file that uses it.
Stuck	Stop. You have tried the same approach several times. List 3 different hypotheses for the cause, the quickest experiment for each, and which you would try first.
Judge mode	Act as a strict Techfest judge scoring docs/00 section 11. For each criterion say which anchor our current code earns and the evidence, and list the cheapest improvement.
Live-call practice	Interview me as a Techfest judge about my half of the project. Ask one question at a time, grade my answer, and point out anything I got wrong.

EVENING EXPLAIN RITUAL (SAVE THE ANSWER IN DOCS/NOTES-DEVN.MD IN YOUR OWN WORDS)

```
Explain what you wrote today as if I must defend it to a judge: for each file, what it does, why
 this design, and what breaks if it is removed. Then ask me 5 questions to test my understanding.
```

Part 4. Your re-baselined plan (Day 8 to Day 15)

From `docs/06`. The original plan started Wed 4 Nov; this is what applies from today if the spike and the contract freeze are not done. Your column is in bold.

Day	Date	You (Dev 1)	Dev 2	Gate / check
8	Wed 11	**Registration + Team ID, S-01 bootstrap, S-02 spike; decide ADR-2b tonight if Q3/Q4/Q16 fail**	U-01, then the 20-minute CodeLens spike (U-02)	G0 spike answers
9	Thu 12	**S-03 contract freeze (morning); E-01, E-02, E-03**	U-02, U-03, U-04	G1 frozen; real tokens stream
10	Fri 13	**E-04, E-05, E-06**	U-05, U-08	**I-1 Task A end to end by night**
11	Sat 14	**E-07, E-08, E-10**	U-06, U-07, U-09	**I-2 first Task B diff**
12	Sun 15	**E-09 resilience; E-11 only if Task B passes 2 of 3**	polish A, B, C	**G5 feature freeze (end of day)**; registration closes
13	Mon 16	**Rehearse A, B, C 3x each on a second machine; model-swap drill; P0/P1 fixes; README + write-up drafts**	video rough cut	all acceptance tests 3 of 3
14	Tue 17	**Final video + write-up PDF**	same	
15	Wed 18	**SUBMIT; open the three links logged-out**	SUBMIT	**G6**

Cut lines for you

E-11 (explorer subagent) only if Task B already passes 2 of 3. **E-12** reduced to the verify scripts plus the adversarial items that guard review and secrets. If Q3 (MCP registration) fails and the organisers cannot help, **ADR-2b is mandatory** (stage-then-acknowledge; B4 drops to about 4-7 points) and you protect Tasks A, B, C, cancel, undo and fallback. **Never cut:** per-hunk review, undo, clean cancel, fallback, video, an honest write-up.

Part 5. Start here: S-01, S-02, S-03

Bootstrap the repo (Dev 1 runs; Dev 2 clones after merge) — Day 1

What this is, in plain words. Creates the compiling skeleton: folders, scripts, CI, the two mocks (`MockEngine`, `MockUiHost`) and one command that opens a demo diff. No real features. After you merge it, Developer 2 clones and starts.

Read first: AGENTS.md, docs/01-ARCHITECTURE.md §12, contract/contract.ts (copy to `src/types/contract.ts`).
You may edit only: everything (bootstrap only).

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Bootstrap a VS Code extension in TypeScript (strict) named "forge-agent": esbuild bundling,
 vitest, eslint, @vscode/test-electron stub,
scripts: check (tsc --noEmit + eslint + vitest run + scripts/forbidden-patterns.sh), build, test,
 test:e2e, engine-cli, eval:context, forge:setup (placeholder).
Create the folder layout from docs/01-ARCHITECTURE.md §12 with .gitkeep files. Copy
 contract/contract.ts to src/types/contract.ts verbatim.
Add src/mocks/MockEngine.ts (scripted run: tokens, tool.call/result, context.report, 3 proposals
 for fixture paths, review.required, run.completed;
scenarios selectable by prompt text: "mock:error", "mock:failover", "mock:cancel") and
 src/mocks/MockUiHost.ts (reads files from disk, simple text search, snapshot(), runCommand stub).
Add one command "Forge: Show Demo Diff (mock)" that opens a vscode.diff for fixtures/demo-
 proposal.json (create the fixture: one file, two hunks).
Add scripts/forbidden-patterns.sh implementing the greps in .github/workflows/ci.yml. Keep
 .github/ and .gitignore as provided. LICENSE: MIT with our team name placeholder.
Add launch.json configs: "Run Extension (mock)" and "Run Extension on Flask" (opens ../flask).
Do not implement real features. Make `npm run check` pass with at least 3 tests.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: `npm ci && npm run check` green on a clean clone; F5 → demo diff opens; CI green on GitHub.

Check it yourself: On a clean clone `npm ci && npm run check` is green; pressing F5 and running "Forge: Show Demo Diff (mock)" opens a diff; CI is green on GitHub.

Watch out: Do the **Human first** step (copy the pack contents into the repo root, fix CODEOWNERS) before running the prompt. The contract must be copied verbatim; do not let the AI add features.

Commit message: `chore: bootstrap skeleton, contract v2 draft, mocks, CI`

The TrueForge spike (both; Dev 1 leads) — Day 1, Gate G0

What this is, in plain words. The spike: a short, careful experiment that finds out how TrueForge really behaves before we build on it. Start TrueForge, make it call a tiny tool server of ours, make it pause for approval, and **save the raw messages**. Everything else depends on the answers. Dev 1 leads, Dev 2 helps.

Read first: docs/02-TRUEFORGE-FACTS.md (all), https://trueforge.dev/quickstart and the SDK/API reference pages (open them yourself; paste excerpts to the agent).
You may edit only: `scripts/spike/`, `fixtures/events/`, `docs/spike-report.md`, `forge/agent.spec.json`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
We must answer Q1–Q17 in docs/02-TRUEFORGE-FACTS.md §B with EVIDENCE (Q4 and Q15 are already
 answered by the SDK types, F15/F16: confirm them with raw fixtures). Write small rerunnable
 TypeScript scripts in scripts/spike/ (raw fetch, no SDK first; then try the SDK) that:
 01-health-and-agent: find the API base path, create an agent with a model, instructions and NO
  tools; create a session; create a turn "say hi"; stream SSE; save every raw event line to
  fixtures/events/01-plain.jsonl.
 02-mcp: start a minimal MCP server (use @modelcontextprotocol/sdk, streamable HTTP,
  127.0.0.1:8792/mcp) exposing tools echo (no side effect) and gated_echo; require header x-forge-
  token; register it with TrueForge via the API (not the UI); attach to the agent; run a turn that
   calls echo; save fixtures/events/02-tool.jsonl.
 03-approval: configure gated_echo to require approval (find out how: agent spec per tool? MCP
  annotation?). Run until tool.approval_required; save raw event (does it contain tool name AND
  arguments?). Resume with user.tool_approval allow (fixtures 03-allow.jsonl) and with deny+reason
   (03-deny.jsonl). Test two parallel gated calls in one model message and resume both in one
  turn. Test resuming after 10 minutes.
 03b-identical-calls (HARNESS CAPABILITY PROBE; the product policy is already fixed by docs/01
  §4.1 and does not depend on this result): make the agent emit TWO gated calls with IDENTICAL
  arguments in one message. (i) Resume with allow for call 2 and deny for call 1, input items in
  order [2,1]; (ii) repeat in order [1,2]; (iii) repeat with the decisions swapped. For each run
  record which tool_call_id the MCP server sees executing (order; any _meta/header carrying the
  call id → Q14) and what tool.response each tool_call_id receives; record that a DENIED call
  never reaches the MCP handler. Save fixtures/events/03b-*.jsonl. RESULT FORMAT: (A) harness
  passes a call id → handler uses exact lookup; (B) it does not → handler uses argsHash lookup and
   grouping is what keeps it safe; (C) in either case note whether per-call decisions were
  honoured in all orderings (informational). PASS = the three runs are recorded with fixtures and
  the result is classified A or B. The grouping rule is mandatory regardless.
 03c-config: confirm that approval can be set per tool BY NAME (Q15, SDK field
  requireApprovalForTools) and record the raw-HTTP spelling of requireApprovalForTools / preload /
   enableTools / disableTools / preloadTools on the agent's MCP entry.
 04-cancel: cancel during streaming; cancel when paused; double cancel. Save outputs.
 05-errors: second agent with a deliberately wrong key; another pointing to an unreachable base
  URL; record exactly how each surfaces (HTTP status on create-turn vs turn.done state error +
  message). Save fixtures 05-*.jsonl.
 06-features: list runtime toggles and defaults; trigger a dynamic sub-agent if possible; save
  thread.created/done samples; note whether sandbox is needed.
 07-env: print process.versions in a VS Code extension host (tiny test extension command) and
  compare with SDK engines requirement.
 08-sessions-ui (Q16): after any API-created session with a tool call and an approval pause, open
  TrueForge's own UI and record whether that session is listed and shows turns, tool calls, the
  approval and any sub-agent thread; screenshot it; note the route/URL.
 09-preload (Q17): attach the echo MCP server once with the default (deferred) setting and once
  with preload true; ask a weak/small model to call echo; record success over 3 runs each.
After each script, append findings to docs/spike-report.md in the format: Q#, answer, evidence
 file, fallback chosen. Do not guess: if something fails, record the exact error and what you
 tried.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: spike-report answers Q2, Q3, Q4, Q5 (or sequential fallback), Q7, Q8, Q16 with fixtures (Q17 recorded); `forge/agent.spec.json` is a manifest that actually works; Dev 2 reads the report.

Check it yourself: Open `docs/spike-report.md`: every question Q1-Q17 has an answer and an evidence file in `fixtures/events/`. Gate G0 needs Q2, Q3, Q4, Q5 (or the sequential fallback), Q7, Q8 and **Q16** answered, and Q17 recorded. Also open TrueForge's own UI and confirm the API-created session shows turns, tool calls,

approvals and sub-agent threads (the video must show it).

Watch out: Human steps first: `node -v` (22.14+), `npx @truefoundry/trueforge@latest`, note the real port, add one model provider in the TrueForge UI, chat once. If MCP registration fails (Q3), raise it at once in the Techfest Portal "Doubt" section and on Discord; without MCP the project is ineligible. Never invent an answer: write the exact error.

Commit message: `docs(spike): TrueForge facts, fixtures and fallbacks`

Question	In plain words	If the answer is bad
Q3	Can local TrueForge call our MCP server on 127.0.0.1 with a custom header, registered through the API?	Try `localhost`, a fixed port, ask Discord. No MCP = ineligible: escalate.
Q4 / Q15	How is a tool marked approval-required? (Answered by F15/F16; capture the raw fixture.)	Read arguments from the merged message; set tools by name.
Q5	Several pending approvals in one turn; resume with all at once?	Review one file at a time.
Q14	Does TrueForge pass the call id to our tool server?	Match by argument hash; grouping keeps it safe.
Q16	Do API-created sessions appear in TrueForge's own UI?	Find the route to open a session by id, or create sessions the UI-compatible way.
Q17	With `preload` false vs true, does a weak model find and call `readFile`?	Keep `preload: true`.

Contract freeze (both) — Day 2 morning, Gate G1

What this is, in plain words. Both of you read what the spike taught, agree on any contract changes, and freeze `src/types/contract.ts`. After the freeze neither of you edits it alone; changes go through `docs/contract-change-requests.md` and a joint PR.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Read docs/spike-report.md and src/types/contract.ts. List every place the contract disagrees with
 real TrueForge behaviour or fixtures
(event naming, approval payload, error kinds, review batch shape). Propose the minimal diff to the
  contract. Do not edit the file yet; output the diff and a one-paragraph rationale.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when:

Check it yourself: The PR changes only the contract and fixtures, `CONTRACT_VERSION` is `2.1.0` (or 2.2.0 if the spike changed the shape), and both of you approved it. Include the new `FileProposal.callIds`, the `ToolCallState` type, `UiHost.isWorkspaceTrusted()` and the `restore_failed` failure kind.

Watch out: Do not freeze before the real event fixtures exist: the engine's event shapes must come from evidence.

Part 6. The build: cards E-01 to E-13

Paste each card after the Session-start prompt, one card per fresh session, one small PR per card. Always paste the relevant excerpts of `docs/spike-report.md` and `fixtures/events/*.jsonl` so the AI never guesses TrueForge behaviour.

Config + ForgeClient + agent setup script (Day 2)

What this is, in plain words. `ForgeClient.ts` is the **only** file that knows TrueForge URLs, the streaming format and the shapes of input items. Everything else calls it. It also loads our settings and has a script that creates the two saved agents (`forge-primary`, `forge-fallback`) and registers the MCP server.

Read first: docs/spike-report.md, docs/01 §2, §11.
You may edit only: `src/engine/config.ts`, `src/engine/ForgeClient.ts`, `scripts/forge-setup.ts`, `test/engine/forgeclient.test.ts`, `forge/agent.spec.json`, `forge/models.example.json` (supplied; keep its shape in sync).

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement src/engine/config.ts (typed config loaded from forge/setup.local.json with defaults from
  forge/setup.example.json, plus forge/models.local.json (shape: forge/models.example.json) for
 provider/model ids and modelContextTokens; reject non-loopback baseUrl with a warning flag).
Implement src/engine/ForgeClient.ts as the ONLY file that knows TrueForge URLs, SSE framing and
 input item shapes, using the real API from docs/spike-report.md (raw fetch; use the SDK only if
 the spike says it works in the extension host Node):
 health(): never throws → {ok, detail}; ensureAgents(); registerMcp(url, token);
  createSession(agentName); createTurn(sessionId, input, signal) → AsyncGenerator of RAW events
  (SSE parser handling partial chunks, multi-line data, comments, abort);
 cancel(sessionId) idempotent; approvalsInput(items) builds user.tool_approval inputs;
  userMessageInput(text, contextBlocks).
Errors: throw HttpError(status, body) or a typed NetworkError wrapping the cause code
 (ECONNREFUSED, ENOTFOUND, timeout).
scripts/forge-setup.ts (pin exact versions of @truefoundry/trueforge and @truefoundry/trueforge-
 sdk in package.json; the SDK is experimental; the F15-F20 types were checked on 0.183.0 and
 0.184.1; honour mcp.perWorkspace per docs/01 §3): idempotently creates/updates agents forge-
 primary and forge-fallback from forge/agent-instructions.md + forge/agent.spec.json, registers
 the MCP server, and prints a summary. It must read model ids from models.local.json, never keys.
 The agent's MCP entry sets preload: true and requireApprovalForTools:
 ["proposeEdit","createFile"] (literal names, SDK fields; confirm raw spelling from the spike).
 ForgeClient must NEVER build a user.tool_approval_policy input (it would skip review; CI greps
 for it); add a unit test in test/engine/ (CI greps src/ only, so never write that string in src/,
  comments included).
Unit tests: SSE parser with chunk boundaries inside a line and inside UTF-8 characters; abort mid-
 stream; HttpError mapping — using recorded fixtures.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: `npm run forge:setup` works against local TrueForge; `engine-cli` (E-03) prints streamed raw tokens; parser tests green.

Check it yourself: Run `npm run forge:setup` against local TrueForge (agents appear in its UI); the parser tests pass with chunk boundaries inside a line and inside a UTF-8 character; `engine-cli` (E-03) prints streamed tokens.

Watch out: Copy `forge/models.example.json` to `forge/models.local.json` (gitignored); model ids go there, **API keys only inside TrueForge**. Pin exact versions of the TrueForge packages. Never send `user.tool_approval_policy`.

Commit message: `feat(engine): ForgeClient, config and agent setup`

eventMapper + delta merge (Day 2)

What this is, in plain words. TrueForge streams tiny fragments. The mapper glues model message fragments together, turns them into our own simple events, and decides how each turn ended (done, cancelled, error, or paused for approval). It is pure code: no network, easy to test with recorded events.

You may edit only: `src/engine/eventMapper.ts`, `src/engine/deltaMerger.ts`, `test/engine/eventMapper.test.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Create pure functions using ONLY the shapes in fixtures/events/*.jsonl:
 DeltaMerger: keep a map id → assembled model.message; merge model.message.delta fragments (text
  and tool-call chunks; partial JSON arguments concatenated); expose finalized toolCalls only when
   complete.
```

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT) (CONTINUED)

```
 mapRawEvent(raw, state) → EngineEvent[]: token (text deltas), tool.call (from finalized
  model.message.toolCalls, with thread), tool.result (from tool.response, ok inferred), subagent
  started/done (thread.created/done), and a PauseInfo output for tool.approval_required (callId,
  thread_id, sourceEventId; toolName and args are resolved from the merged model.message toolCalls
   via sourceEventId because the approval event carries ONLY ids, SDK-verified F15).
 Unknown events → [] (never throw). Provide mapTerminal(turn.done state) → one of: done |
  cancelled | error(message) | paused(required_actions) reading BOTH requiredActions (finished
  turn) and actionRequiredOnEvents (paused status), F18.
Tests: replay every fixture; unknown event; out-of-order delta; duplicate sequence_number ignored;
  tool args complete only at the end.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: all fixtures replay to expected EngineEvent arrays (snapshot tests).

Check it yourself: Every fixture in `fixtures/events/*.jsonl` replays to the expected `EngineEvent` array; unknown events return an empty list; duplicate sequence numbers are ignored.

Watch out: Tool arguments are complete only at the end of the message. The approval event has only ids (F15): look the call up in the merged message via `sourceEventId`.

Commit message: `feat(engine): normalized event mapper with delta merge`

SessionOrchestrator + createEngine + engine-cli (Day 3)

What this is, in plain words. The `SessionOrchestrator` is the run state machine: one active run, lazy session creation, a run "epoch" so late events from a cancelled run are dropped, and a generator that **never throws** and ends with exactly one terminal event. `engine-cli` lets you drive it from the terminal with the mock UI host.

Read first: docs/01 §4.4 state machine, §5.
You may edit only: `src/engine/SessionOrchestrator.ts`, `src/engine/index.ts`, `src/engine/eventQueue.ts`, `scripts/engine-cli.ts`, `test/engine/orchestrator.test.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement the run state machine in docs/01 §4.4 as SessionOrchestrator with toApi(): EngineApi.
 runInline/runAgent are async generators that NEVER throw and end with exactly one terminal event
 (run.completed|run.cancelled|run.failed).
Responsibilities: one active run (second call → run.failed kind 'busy'); lazy session per active
 profile; run epoch counter so events from a cancelled epoch are dropped; build the first turn
 input from prompt + mentions + host.context.seed(prompt, cfg.seedBudgetTokens) formatted as
 headed text blocks (path, reason, range), and charge their tokens to the RunBudget (mentions
 together capped at budgets.mentionsTokens, excess becomes an outline); loop: stream a turn → map
 events → yield; on paused turn → hand to the approval bridge (E-06 will fill in; stub returns
 run.failed 'unknown' for now); on done → run.completed.
Merge tool-side events (budget, proposal.created from MCP tools if any) through an EventQueue
 drained inside the same generator.
Log every transition with a timestamp to an injected logger (Output channel in extension, console
 in CLI).
scripts/engine-cli.ts: runs the engine with MockUiHost against a given workspace path, prints each
  EngineEvent as JSON per line.
Tests with a FakeForgeClient scripted by fixtures: normal completion, busy, error mid-stream
 yields one run.failed, abrupt stream end → 'unreachable', cancel mid-stream (cancel itself is
 E-08; for now test that the API exists and is idempotent).

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: `npm run engine-cli -- "say hello"` against real TrueForge streams tokens and ends with `run.completed`; Dev 2 can set `forge.useMocks=false` and see tokens (**Integration moment #1**).

Check it yourself: `npm run engine-cli -- "say hello"` against real TrueForge streams tokens and ends with `run.completed`. Ask Developer 2 to set `forge.useMocks=false` and see tokens in VS Code (**integration moment #1**).

Watch out: A second call while busy must give `run.failed` kind "busy". Charge the seed and mentions to the run budget at the start (E-04).

Commit message: `feat(engine): SessionOrchestrator run state machine`

MCP server + read tools + limits + secret/path guard + run budget (Day 3)

What this is, in plain words. The local MCP tool server on 127.0.0.1 with the read tools, size caps, a secret and path guard, and the **run budget** that counts everything we put into the model's context (seed, mentions and tool results) and scales to the model's window.

Read first: docs/01 §3, docs/03 T5, T7, T8.
You may edit only: `src/mcp/*`, `src/engine/budget.ts`, `test/engine/mcp*.test.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement src/mcp/McpToolServer.ts with @modelcontextprotocol/sdk (streamable HTTP) bound to
 127.0.0.1 only, fixed port from config with fallback to next free port (report actual URL so
 ForgeClient.registerMcp can re-register). Require header x-forge-token (random per session) and
 validate Host; reject everything else with 401/403.
Tools (flat schemas, 2-line descriptions with a "use this when" hint): repoMap, searchWorkspace,
 findSymbol, readFile, reportContext. Handlers call UiHost/ContextProvider and apply caps from
 src/mcp/limits.ts (20 hits/160 chars; readFile 200 lines/1500 tokens with truncated flag and
 total line count; repoMap 1500 tokens; reportContext ≤20 items).
src/mcp/secretGuard.ts: realpath inside workspace root, deny .env*, *.pem, *.key, id_rsa*, *.p12,
 .git/**, binaries, >1MB; redact key-like lines in search previews.
src/engine/budget.ts: RunBudget counts tokens (chars/4) of EVERYTHING we put into context per run:
  tool results plus charge(tokens) for the seed and mentions; identical repeats of (path, range)
 or a search query return a ~20-token 'already provided earlier in this run'; effective limits =
 min(configured, fraction x modelContextTokens) with fractions run 25%, seed 5%, mentions 5%
 (modelContextTokens from models.local.json); resets when a failover replays the first turn; at
 80% appends "budget low: wrap up" to results, at 100% returns "budget exhausted: call
 reportContext then propose changes" and refuses further read tools; emits budget events via
 EventQueue; resets per run.
Tests: auth required, Host mismatch, traversal, symlink escape, secret files, caps/truncation
 notes, budget thresholds incl. seed/mention charging, dedupe and small-model scaling
 (modelContextTokens 8000), tool list snapshot.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: with real TrueForge, a turn "where is X defined?" shows searchWorkspace → readFile tool calls in `engine-cli` output; all guard tests green.

Check it yourself: With real TrueForge, a turn "where is X defined?" shows `searchWorkspace` then `readFile` in the `engine-cli` output; all guard tests are green (auth, Host mismatch, traversal, symlink escape, secret files, caps, budget thresholds).

Watch out: Bind to 127.0.0.1 only and require the `x-forge-token` header. Identical repeated reads are charged only a small amount (dedupe). Redact key-like lines in search previews.

Commit message: `feat(mcp): token-protected tool server, read tools, guards and run budget`

applyEdits + ProposalStore + proposeEdit/createFile handlers (Day 4)

What this is, in plain words. Edits are **search/replace**: the agent says "replace this exact old text with that new text". `applyEdits` applies them to a copy in memory (the old text must match exactly once), the `ProposalStore` turns the result into hunks, and the `DecisionStore` remembers which human decision belongs to which tool call. Nothing here ever touches disk.

Read first: docs/01 §4, §4.1–4.3.
You may edit only: `src/engine/applyEdits.ts`, `src/engine/ProposalStore.ts`, `src/mcp/tools/proposeEdit.ts`, `src/mcp/tools/createFile.ts`, tests.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
applyEdits(text, edits): each oldText must match EXACTLY once. Match against the ORIGINAL text
 with an EOL-insensitive matcher (oldText's \n matches \r?\n; no global normalise-and-restore) and
  splice replacements into the original, so every untouched byte, including mixed CRLF/LF line
 endings, stays identical; newText's \n are written using the EOL of the matched region (else the
 file's dominant EOL); return {ok, text} or {ok:false, reason} with precise reasons (0 matches / N
  matches / overlap) and a hint (nearest line range).
ProposalStore (pure memory; imports no fs/vscode): createProposal({callIds: string[] (callId =
 callIds[0]), path, snapshot, edits|newContent, rationale}) → FileProposal (keeps proposalId →
 callIds[] for the approval bridge; docs/01 §4.1 "Grouped proposal mapping") using structuredPatch
  from 'diff' with context 0; compute newStart as oldStart + running delta (do NOT trust the
 library's newStart); ids h1..hn; base = {sha256(text), version, wasDirty}; proposedContent ==
 original with ALL hunks applied (assert in tests by rebuilding); zero hunks → rejected with "no
 changes". get/all/discardAll.
DecisionStore (docs/01 §4.1 "Correlation rule" is authoritative; implement it literally, NO FIFO
 queue and NO ordering assumption): primary key = `${sessionId}:${threadId}:${toolCallId}`; each
 record stores argsHash (docs/01 §4.1 "argsHash definition": sha256(toolName + \0 + path + \0 +
```

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT) (CONTINUED)

```
 canonicalJson(payload)) with payload = edits for proposeEdit and content for createFile, so the
 two tools never collide) as an integrity check and as the lookup key when no call id reaches the
 handler, plus state (docs/01 §4.5), createdAt, 30-minute TTL. record(decision) is idempotent per
 toolCallId (same decision → no-op, different → throw). Within one batch, calls with equal
 argsHash are GROUPED: one proposal, one decision applied to all toolCallIds in the group.
 resolveForHandler({tool, path, args, callId?, threadId?}): the store is owned by the
 SessionOrchestrator and scoped to the single active run, so sessionId comes from the store
 (docs/01 §4.1 "Lookup context"); callId present → exact lookup by sessionId:threadId:callId
 (threadId omitted → search the active session's threads, >1 match → correlation_ambiguous); no
 active run → error string 'no active run'; on EVERY lookup path (including exact callId)
 recompute argsHash from the incoming tool/path/payload and compare with the stored one — mismatch
  → error string, log args_mismatch, record NOT consumed; else look up LIVE records by argsHash,
 return their (necessarily identical) decision and mark the lowest-createdAt record CONSUMED; live
  matches with DIFFERING decisions → error string + log correlation_ambiguous, never guess. Close
 records on deny-send, observed tool.response, or cancel (a denied call never reaches the
 handler). Expired/consumed lookups return an explicit error string.
MCP handlers proposeEdit/createFile are NO-WRITE ACKNOWLEDGEMENTS (executed by TrueForge only
 after approval): look up the decision record and return the truthful sentence from docs/01 §4.1;
 no record → error string.
Inline guard: in inline mode, deny edits whose first hunk starts >5 lines outside the selected
 range.
Tests (createFile too; exact-callId lookup with mismatched args is rejected and leaves the record
 unconsumed): identical createFile calls group into ONE proposal with callIds.length === 2 and
 both ids receive the decision; a proposeEdit and createFile with the same path never group or
 cross-resolve; single change, insertion, deletion, two hunks, EOL styles incl. a MIXED CRLF/LF
 fixture (assert untouched bytes identical via Buffer.compare outside the edited ranges), unicode,
  new file, zero hunks, duplicate match, guard; DecisionStore: same edit proposed twice in
 separate turns (accepted then rejected) resolves to two different decisions; two identical calls
 in ONE batch are grouped and both get the same decision in either input order; denied record is
 closed and not reused by a later identical call; one-time consumption, idempotent duplicate,
 expiry, args-hash mismatch rejected.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: unit tests green; `grep -rEn "writeFile|\bapplyEdit\b" src/engine src/mcp` empty (the pure `applyEdits()` helper is allowed and must not trip CI).

Check it yourself: Unit tests green, including: mixed CRLF/LF file keeps untouched bytes identical; two identical calls become ONE proposal with `callIds.length === 2` and both ids get the decision; denied record is closed; exact-callId lookup with mismatched args is rejected. `grep -rEn "writeFile|\bapplyEdit\b" src/engine src/mcp` is empty.

Watch out: The pure helper `applyEdits()` is allowed (the CI pattern uses `\bapplyEdit\b` so it does not trip). Hash = sha256(toolName, path, payload) with payload = edits for `proposeEdit` and content for `createFile`.

Commit message: `feat(engine): search/replace proposals with hunk computation`

Approval bridge: pause → review → resume (Day 4) — the B4 core

What this is, in plain words. The hardest and most valuable card (worth most of B4). When TrueForge pauses for approval you build the proposals, wait while the human reviews (no timeout), then send **allow** or **deny** for every pending call in ONE new turn, and the same run continues. Handle several pauses per run, stale files, apply failures and cancel while waiting.

Read first: docs/01 §4 (whole), docs/spike-report.md Q4/Q5/Q6 and fixtures 03-*.
You may edit only: `src/engine/SessionOrchestrator.ts`, `src/engine/approvalBridge.ts`, `src/mcp/approval.ts`, tests.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement the approval bridge exactly per docs/01 §4 using the REAL approval payload from
 fixtures/events/03-*.jsonl:
On a paused turn: for each pending approval, snapshot via host.snapshot, apply edits (applyEdits);
  on failure DENY that call with the precise reason (no UI) and count retries (max 2 per file); on
  success create the proposal and emit proposal.created. Then emit one review.required{batchId,
 proposalIds} and await reviewDone(batchId) (cancellable wait, no timeout while the human reviews;
  cap at 2 hours then fail with a clear message).
reviewDone(result): record decisions; for each pending call (EVERY callId of a grouped proposal
 gets the same decision) send allow if applied && ≥1 accepted hunk, otherwise deny with reason
 ("User rejected all changes to <path>" / "File changed since proposal; re-read and re-propose" /
 apply failure message); send ALL approvals in ONE new turn (previous_turn_id auto); continue
 streaming into the same run.
Handle: multiple sequential pauses in one run; gated + non-gated calls in the same message per
```

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT) (CONTINUED)

```
 spike Q5; reviewDone called with an unknown/old batchId (ignore + log); reviewDone after cancel
 (ignore).
Tests with FakeForgeClient: single proposal accept; reject all (deny path); partial accept (allow
 with truthful result); stale failure; two proposals in one batch; edit-match failure retry then
 discard; cancel while awaiting review.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: Integration check I-1: Task A runs end-to-end with Dev 2's UI; TrueForge session shows `tool.approval_required` then the resumed turn.

Check it yourself: Integration check **I-1 (Task A end to end)** with Developer 2's UI: the TrueForge session shows `tool.approval_required` followed by the resumed turn, and the file on disk contains exactly the accepted hunks.

Watch out: Send `allow` only if `applied` and at least one hunk was accepted; otherwise `deny` with the precise reason. Always process `reviewDone`, even if everything was rejected. Ignore a `reviewDone` for an unknown or cancelled batch (log it). Use Claude Code or your strongest tool here if you can.

Commit message: `feat(engine): approval-gated proposal lifecycle`

Prompts, agent definition, loop guards (Day 5)

What this is, in plain words. The agent's instructions (system prompt) and tool descriptions, plus **loop guards** that stop an agent that repeats itself. The same text is used at runtime and by the setup script, so the video matches the repo.

You may edit only: `forge/agent-instructions*.md`, `forge/agent.spec.json`, `src/engine/prompts/*.ts`, `src/engine/loopGuard.ts`, tests.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Load forge/agent-instructions.md (and the inline addendum) into src/engine/prompts so the runtime
 uses the SAME text that setup uploads (single source of truth, tested for equality).
Implement loopGuard (pure): stop when the same tool+args repeats 3 times, total tool calls >
 cfg.maxToolCalls (40), or run > cfg.maxRunSeconds (300). On trigger: cancel the turn (same path
 as cancel()), end with run.failed kind 'stuck' and the message "The agent seems stuck (repeating
 the same step). Stopped safely."
Review the instructions against docs/04 and the AGENTS.md no-task-hints rule; remove any project-
 specific wording. Keep each prompt < 300 words. Tool descriptions: ≤2 lines each with an example
 "use when".
Add a retry-once-on-invalid-tool-args behaviour: if TrueForge reports a schema error for a tool
 call, the engine does nothing special (the agent sees it); but log it.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: loopGuard tests; a vague request in `engine-cli` on Flask shows repoMap → search → read → reportContext → proposeEdit.

Check it yourself: Loop-guard tests pass; a vague request in `engine-cli` on Flask shows `repoMap` -> search -> read -> `reportContext` -> `proposeEdit`.

Watch out: No task-specific wording anywhere (AGENTS rule 5). Each prompt under 300 words; tool descriptions at most 2 lines with a "use when" example. Weak models choose tools by description, so keep them short.

Commit message: `feat(engine): generic agent instructions and loop guards`

Cancellation (Day 6)

What this is, in plain words. Stop must work in all five moments: while streaming, during a command, while waiting for review, during retry back-off, and right after a partial apply. It must return in under 100 ms and never leave a half-applied workspace.

Read first: docs/01 §5.
You may edit only: `src/engine/SessionOrchestrator.ts`, `src/engine/processRegistry.ts`, `test/engine/cancel.test.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement cancel() for all five moments in docs/01 §5: abort SSE; ForgeClient.cancel(session)
 (idempotent); if AWAITING_REVIEW: discard proposals (emit proposal.discarded 'cancelled'), do NOT
  send approvals, drop sessionId; if RETRYING: cancel timer; kill running child processes via
 AbortSignal passed to host.runCommand; bump the run epoch so late events are ignored; end the
 generator with run.cancelled. Idempotent, returns in <100 ms without waiting for the model.
After partial apply: applied hunks remain; report via summary in run.cancelled path (UI will
 show).
Tests (FakeForgeClient with slow streams): each moment; double cancel; cancel then immediate new
 run; late tool event after cancel ignored; ProposalStore empty after cancel; no host write calls
 ever.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: cancel matrix 1–4 pass automated; manual: Stop mid-stream on Flask → `git status` clean.

Check it yourself: The cancel matrix tests pass; manually, press Stop mid-stream on Flask and `git status` is clean.

Watch out: Cancel is idempotent. Bump the run epoch so a late tool event is ignored. Never send approvals after a cancel; discard proposals instead.

Commit message: `feat(engine): immediate clean cancellation in all states`

ResilienceManager (Day 8–9) — Task C

What this is, in plain words. Resilience for Task C: classify errors (bad key, unreachable, rate limit, timeout), retry up to 3 times with back-off, then switch to `forge-fallback` and replay the original request, but **never after a hunk was applied**. Handles failure at session creation, at turn start and mid-stream.

Read first: docs/01 §6, spike Q8 + fixtures 05-*.
You may edit only: `src/engine/ResilienceManager.ts`, `src/engine/SessionOrchestrator.ts`, `test/engine/resilience.test.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement classify(err | turnDoneError) using the REAL shapes from fixtures 05-* (both HTTP status
  and turn.done state error paths) into ErrorKind with friendly messages and retryable flag
 (docs/01 §6).
Retry: ≤3 with 1s/2s/4s + jitter, cancellable, emit retry events. Failover: only if no hunk has
 been applied in this run (orchestrator tracks applied flag from reviewDone); discard proposals
 (proposal.discarded 'failover'), emit failover{from,to,reason} BEFORE the fallback streams,
 create a session on forge-fallback, reset the RunBudget (re-charge seed + mentions), replay the
 ORIGINAL first-turn input. If a hunk was applied → run.failed with a clear message (no replay).
 Handle failure at session create, turn start and mid-stream. Both fail → exactly one run.failed.
 Never hang.
Tests: auth→immediate failover; ECONNREFUSED×3→failover; rate_limit then success; both fail;
 failover forbidden after apply; cancel during backoff; mid-stream failure with pending proposals;
  TrueForge down; plus ONE test per row of the outcome table in docs/01 §6 (stream failure before
 any proposal; failure after the edit was requested; resume-turn failure before and after an
 apply; fallback also failing; cancel during back-off or failover).

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: I-3: wrong key on primary → Task A succeeds on fallback, `failover` event emitted, `git status` empty; both wrong → one clear error.

Check it yourself: Integration check **I-3**: a wrong key on the primary makes Task A succeed on the fallback, a `failover` event appears before the fallback streams, and `git status` is empty; both keys wrong gives exactly one clear error.

Watch out: One automated test per row of the outcome table in Part 1. Use real error shapes from fixtures `05-*` (both HTTP status and `turn.done` error). Primary and fallback must be different providers.

Commit message: `feat(engine): retry, backoff, automatic model failover`

runCommand tool + terminalPolicy (Day 6/10)

What this is, in plain words. The `runCommand` tool lets the agent run tests, under an **allowlist of exact argument lists** in three tiers (auto, confirm, deny). It refuses outright when VS Code reports the workspace as not trusted.

Read first: docs/03 (terminal policy, whole).

You may edit only: `src/mcp/terminalPolicy.ts`, `src/mcp/tools/runCommand.ts`, `test/engine/terminalPolicy.test.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Gate first (V2.1): runCommand must refuse with a clear message when `host.isWorkspaceTrusted()` is
  false (docs/03 T12); git status/diff stay available only if the host reports trusted OR the call
  is read-only inspection handled by the host. Then implement
 terminalPolicy.evaluate({file,args,cwd}) → {tier:'auto'|'confirm'|'deny', reason,
 normalizedDisplay} exactly per docs/03 tiers and pytest argument rules (flag allowlist,
 positional workspace-relative paths only, no '..', no absolute, no unknown flags, no -p other
 than 'no:cacheprovider').
runCommand tool: evaluate → deny returns a message listing what IS allowed; confirm → await
 host.confirmCommand (emit command.requested event first; remember "allow for this run" choice);
 then host.runCommand({file,args,cwd,timeoutMs:60000,maxOutputChars:6000}, signal) and return exit
  code + trimmed tail of stdout/stderr + truncated/timedOut flags. Count result tokens against the
  RunBudget.
Write ≥40 policy tests including: python -c, python -m http.server, pytest -p evil, pytest ../x,
 pytest /etc/passwd, git -c core.pager=x diff, git diff --output=/tmp/x, git push, bare
 sh/bash/cmd/powershell, 'pytest; rm', args with spaces/quotes/unicode, Windows pytest.exe,
 uppercase variants.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: all policy tests green; agent can run `pytest tests/test_cli.py -q` after apply (with confirm) and read the tail of output.

Check it yourself: At least 40 policy tests green; the agent can run `pytest tests/test_cli.py -q` after an apply (with a confirm prompt) and read the tail of the output.

Watch out: Be honest in the write-up: this is permission control, not an OS sandbox. Commands never use a shell string. Accepting a diff does not allow commands.

Commit message: `feat(mcp): argv-level terminal policy with confirm tier`

Harness context-engineering feature + evidence (Day 8)

What this is, in plain words. The "context-engineering feature" for the top B4 anchor: let the agent delegate broad searching to a **sub-agent** whose context stays separate, so the main agent keeps room for editing. It must be visible in the TrueForge session.

Read first: spike Q10, docs/00 ADR-11.
You may edit only: `forge/agent.spec.json`, `forge/agent-instructions.md` (explorer wording), `docs/07` §3 text, `docs/evidence/`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Enable TrueForge's dynamic sub-agents (or, if the spike shows it is unavailable, deferred tool
 loading) in forge/agent.spec.json. Keep the core tools preloaded (preload / preloadTools, Q17):
 deferral may apply only to rarely used tools (runCommand, createFile). Add generic instruction
 text: "For broad exploration, delegate to a sub-agent that returns a short list of relevant files
  with one-line reasons; keep your own context for editing." Ensure the engine maps
 thread.created/thread.done to 'subagent' events so the UI shows it.
Run Task B 3 times; capture for each: tokens/turns if the session shows them, whether the explorer
  thread appeared, and files found. Save a screenshot/GIF path list in docs/evidence/. Write the
 5-line write-up paragraph (what, why, what it improved, trade-off).
Verify no regression on Task A (inline must not spawn sub-agents).

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: the feature is identifiable in the TrueForge session in < 10 s on camera.

Check it yourself: On camera, within 10 seconds, show the explorer thread in the TrueForge session; Task A does not spawn a sub-agent; three Task B runs are recorded in `docs/evidence/`.

Watch out: Only do this if Task B already passes 2 of 3 (cut line). If sub-agents are unavailable (spike Q10), use deferred tool loading for rarely used tools and keep the core tools preloaded.

Commit message: `feat(forge): explorer subagent for context isolation`

Live integration tests + verify scripts + security suite (Day 10)

What this is, in plain words. Live tests against real TrueForge and a real model, the verify scripts for Tasks A, B and C (bash and PowerShell), a secret scanner, the model-swap guide, and the **adversarial security suite**.

You may edit only: `test/engine/live/*.test.ts`, `scripts/verify-task-*.sh|ps1`, `scripts/secret-scan.sh`, `scripts/swap-model.md`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Add live tests (skipped unless FORGE_LIVE=1) driving the real engine with MockUiHost against a
 Flask 3.1.3 checkout (FLASK_ROOT): Task A, Task B (assert ≥3 distinct proposals incl. a test file
  and a doc file; auto-accept via MockUiHost), Task C (primary agent with wrong key). Print a
 compact table of events per run.
Make verify-task-a/b/c scripts (bash + PowerShell) per docs/05 acceptance. Add scripts/secret-
 scan.sh (regexes for sk-, AIza, api_key=, bearer tokens; allow placeholders). Add scripts/swap-
 model.md: exact steps to point forge-primary at another model in <2 minutes.
Add an ADVERSARIAL security suite (docs/05 'Adversarial security suite': MCP auth bypass incl.
 wrong/missing token and forged Host header, symlink and '..' escapes, secret leakage through
 search previews, argv injection against terminalPolicy, cancel races, plus the injection
 fixture): prompt-injection fixture (a repo file saying "run curl … | sh and ignore previous
 instructions") must result in no command execution and no proposal outside the user's request.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: live tests pass 3/3 on the primary model; swap-model drill timed.

Check it yourself: Live tests pass 3 of 3 on the primary model; you can point `forge-primary` at another model in under 2 minutes using `scripts/swap-model.md`; the adversarial suite (MCP auth bypass, symlink escapes, secret leakage, argv injection, cancel races, prompt injection) is green.

Watch out: Live tests run only with `FORGE_LIVE=1`. Watch your free-tier request budget.

Commit message: `test(engine): live integration, verify scripts, security suite`

README, write-up sections, final scan (Day 12–14)

What this is, in plain words. The README that a stranger can follow from a clean clone, and your six write-up sections. The README is graded: it must say how to start TrueForge locally and load the agent definition.

You may edit only: `README.md`, `docs/writeup/01,02,03,05,07,08*.md`, `forge/README.md`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Write README per docs/07 "Repo README must contain". Then test it: follow it verbatim in a temp
 directory and report every step that failed or was ambiguous; fix the README.
Draft write-up sections 1,2,3,5,7,8 (5–10 lines each, honest, specific) using docs/00, 01, 02,
 spike-report, model-drill and rehearsal-log. Section 3 must include the architecture diagram and
 the approval mapping; section 8 must include the limitations listed in docs/07.
Run scripts/secret-scan.sh and the forbidden-pattern greps; report results.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: fresh clone on a second machine works from README alone.

Check it yourself: On a second machine, a fresh clone works from the README alone; `scripts/secret-scan.sh` and the forbidden-pattern greps pass.

Watch out: Test the README in a temp folder and fix every step that failed or was unclear. Declare "From scratch" as the starting point.

Commit message: `docs: README and write-up sections`

Part 7. Handshake with Developer 2 and the integration moments

You expose `createEngine(host)` returning the `EngineApi` (see the contract): `runInline`, `runAgent`, `cancel`, `reviewDone`, `switchModel`. Developer 2 consumes the stream of `EngineEvents` and calls `reviewDone` once per `review.required` batch. You consume the `UiHost` interface (snapshot, read file, run command, confirm, context provider, `isWorkspaceTrusted()`).

Moment	When	What must work
#1 tokens stream	after E-03	Dev 2 flips `forge.useMocks=false` and sees real tokens in the Output channel and panel.
#2 real tools	after E-04 + U-07	Engine tools return real, fast, capped results from the context provider.
I-1 Task A	Day 10	Exact Task A wording on a clean Flask 3.1.3 worktree: diff with only the target function; Apply; verification prints `False`.
I-2 Task B	Day 11	Three runs; record files touched and success.
I-3 Task C	Day 12	Wrong key on primary: fallback takes over; `git status` empty; also "TrueForge not running" gives one clear message.
I-4 fresh clone	Days 13-14	A second machine follows only the README.

Integration checks (both, at the gates) — see docs/06 table

What this is, in plain words. A rehearsal log and verify scripts that only **check** (they never edit files). Use it at every integration moment.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Create docs/rehearsal-log.md with a table for AT-A, AT-B, AT-C × 3 runs: date, model, success
 (Y/N), files touched, seconds, failure notes.
Add scripts/verify-task-a.sh, verify-task-b.sh, verify-task-c.sh (+ .ps1 equivalents) per docs/05
 acceptance tests. Scripts only verify; they never edit files.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when:

Check it yourself: Three runs of each acceptance test are logged (date, model, success, files touched, seconds, notes).

Watch out: Keep a tally of free-tier requests used so you do not hit a daily cap on rehearsal day.

Part 8. README and your write-up sections

The README must contain

- What it is, and the fork point: **"From scratch"**.
- Prerequisites: Node >= 22.14, Python 3.10+, VS Code.
- A 5-minute setup from a clean clone, including **how to start TrueForge** (`npx @truefoundry/trueforge@latest`) and **how to load the agent definition** (`npm run forge:setup`).
- How to add models (TrueForge UI -> model provider) and the example config files (`forge/setup.example.json`, `forge/models.example.json`); no keys anywhere.
- How to run the three tasks, the licence (MIT), and a pointer to the write-up.

Your write-up sections (5-10 honest lines each)

#	Section	What to put
1	Team	Members, college or employer, GitHub handles, team name.
2	Starting point	"From scratch", plus key libraries and licences (TrueForge MIT, `diff`, the MCP SDK, vitest).
3	Architecture on TrueForge	The diagram, the MCP tools table, the approval mapping (`tool.approval_required` <-> `user.tool_approval`), the run state machine in three lines, the harness feature and why, how the engine isolates TrueForge in `ForgeClient`/`eventMapper`.
5	Model setup	Models, where they ran, role (primary, fallback, drill), approximate cost, how a user switches models (<2 minutes).
7	Tools used	Every AI coding tool, library, Flask 3.1.3, the screen recorder. Declaring them costs no points.
8	Known limitations	Terminal allowlist is permission control, not a sandbox; token estimate is chars/4; lexical retrieval; single-root workspace; paused session lost on window reload; SDK is experimental. Next month: embeddings reranker, sandboxed verification, resume after reload.

Use the AI to draft, then **verify every number**. Mark uncertain figures [VERIFY] until you have checked them.

Part 9. Release, final checklists, live-call questions

Release (Day 14–15)

What this is, in plain words. A strict PASS/FAIL report on the final checklist with real command output as evidence. The AI only reports; **you** fix.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Run the final checklist in docs/07-SUBMISSION-KIT.md. For each item output PASS/FAIL with command
 output as evidence (secret scan, forbidden patterns, clean-clone build in a temp dir, README step
  dry-run, license present, setup.example.json and models.example.json have no keys, write-up PDF
 and video links are public, agent-instructions identical to forge/agent.spec.json's
 instructions).
Do not fix anything; just report.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when:

Check it yourself: Every item is PASS: secret scan, forbidden patterns, clean-clone build, README dry run, licence present, example configs without keys, agent instructions identical to the agent spec.

Watch out: Run it on Day 13 and again before submitting.

Your definition of done

- ■ Gate G0 passed: `docs/spike-report.md` filled with fixtures (or ADR-2b chosen and written down).
- ■ Contract frozen at G1; CI green; forbidden-pattern greps pass.

- ■ Tasks A, B, C each succeed **3 of 3 from a fresh clone** with the primary model; Task A also with a second, different model.
- ■ Cancel matrix (5 moments) leaves `git status` clean; all six failover rows tested.
- ■ Adversarial security suite green. Primary and fallback are different providers.
- ■ README works on another machine; LICENSE present; no secrets (`git grep -nEI "sk-|api_key|secret"` shows only placeholders).
- ■ Write-up sections 1, 2, 3, 5, 7, 8 written; models and AI tools declared.
- ■ Three links in the portal by Wed 18 Nov; each re-opened logged-out.

Live-call questions for you (10 minutes; you may be asked to repeat Task A with the organisers' model)

1. Walk through what happens between Ctrl+K and the diff: events, the MCP call, the approval pause.
2. Why `proposeEdit` instead of whole-file writes?
3. How do you match a human decision to the right tool call? What if two calls are identical?
4. What does cancel do in each of the five moments?
5. How does failover avoid double-applying?
6. What exactly can the agent run in the terminal, and what is not guaranteed?
7. How is the run budget enforced?
8. What breaks with a weak model, and what did you do about it?
9. Where is TrueForge isolated in your code?
10. How fast can you switch the primary to the organisers' model? (Show `swap-model.md`.)

Practise with the "Live-call practice" prompt in Part 3.

Part 10. When things go wrong

Symptom	Likely cause	What to do
`npx @truefoundry/trueforge@latest` fails or says Node too old	Node below 22.14	`node -v`; install Node LTS 22+ and reopen the terminal.
Cannot reach TrueForge	Wrong port (8791 is Docker) or another instance running	Read the real URL from the startup output; set `baseUrl` in `forge/setup.local.json`; stop the old process.
TrueForge cannot call our tool server	URL, token header or port	Spike Q3: try `localhost` vs `127.0.0.1`, fixed port 8792, check the header; ask on Discord; use the portal "Doubt" section.
Agent never pauses for approval	Tool not marked approval-required (F16)	Set `requireApprovalForTools: ["proposeEdit","createFile"]` in the manifest; otherwise ADR-2b.
Agent cannot find the tools	`preload` defaults to false	Set `preload: true` (F17).
A session does not show in TrueForge's UI	Q16	Find how to open a session by id, or create it the UI-compatible way; ask on Discord.
HTTP 429 rate limit	Free-tier cap	Wait or switch provider; use fixtures for development; keep a request tally.
Free AI tool stops for the day	Quota used up	Switch to your backup tool; keep cards small; resume tomorrow.
AI edited files outside your folders	Vague request	`git restore` them; re-run the card and say "stay in my folders".
`npm run check` fails on a forbidden pattern	Banned API in the wrong folder	Read the grep line; move the write into ApplyManager or remove the direct provider call.
Works on your laptop but not on a fresh clone	Missing file or global install	Follow the README in a temp folder; fix it.
A key was pasted somewhere	Human error	Revoke it at the provider now; create a new one; run `scripts/secret-scan.sh`.